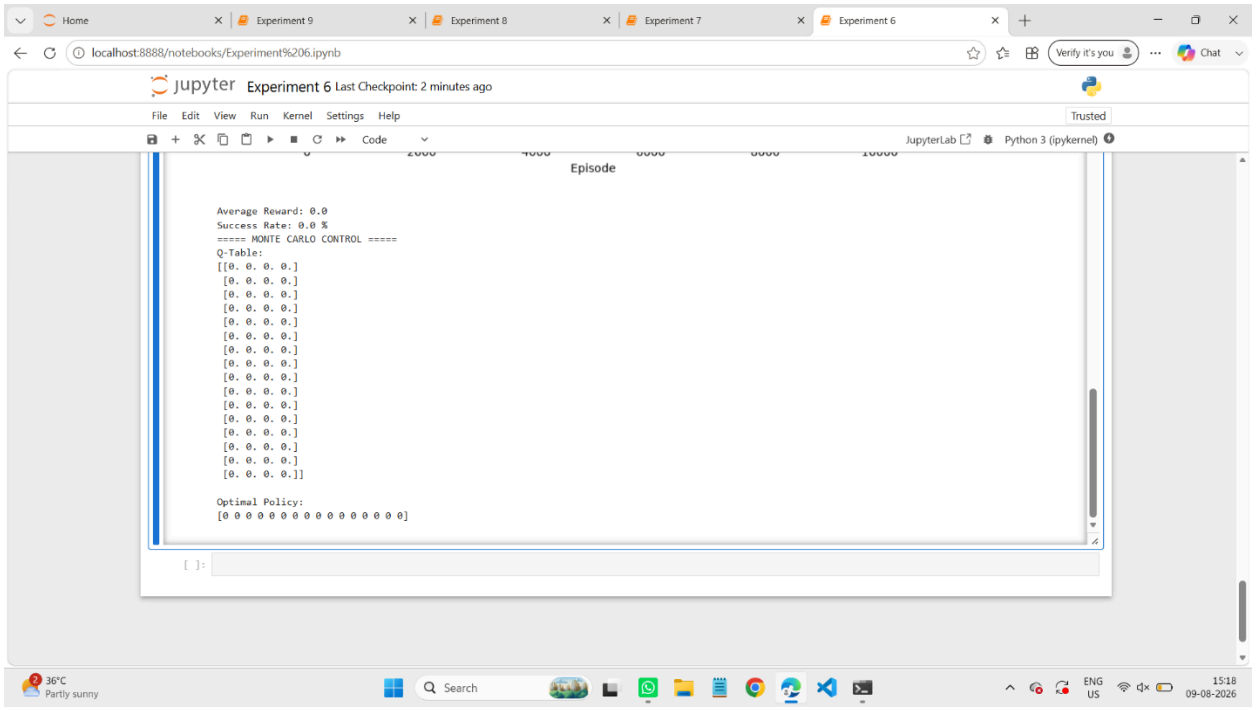
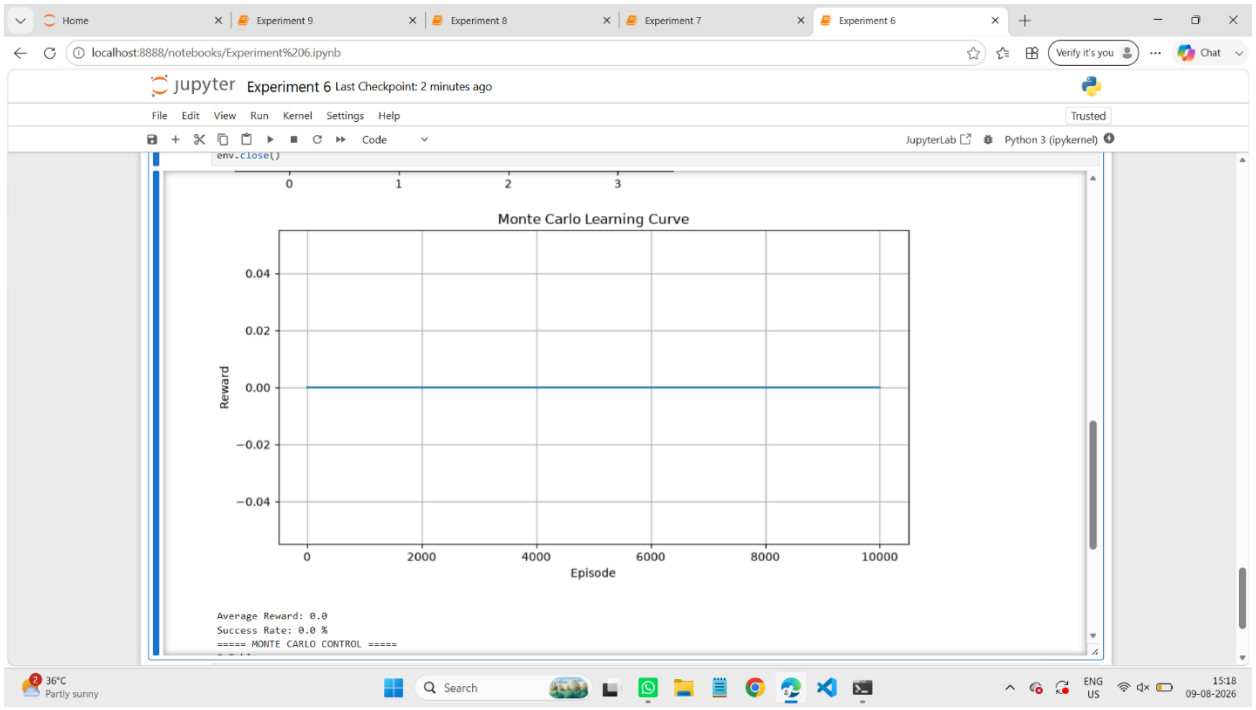
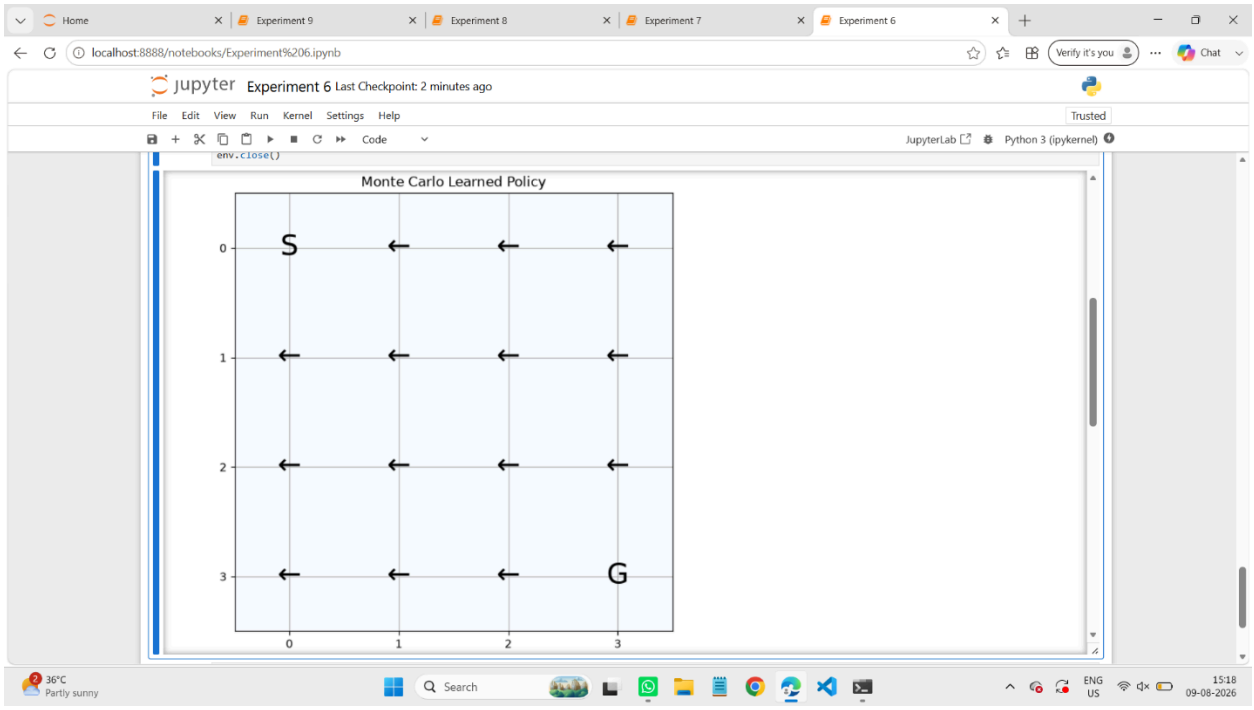
Experiment 5 RL

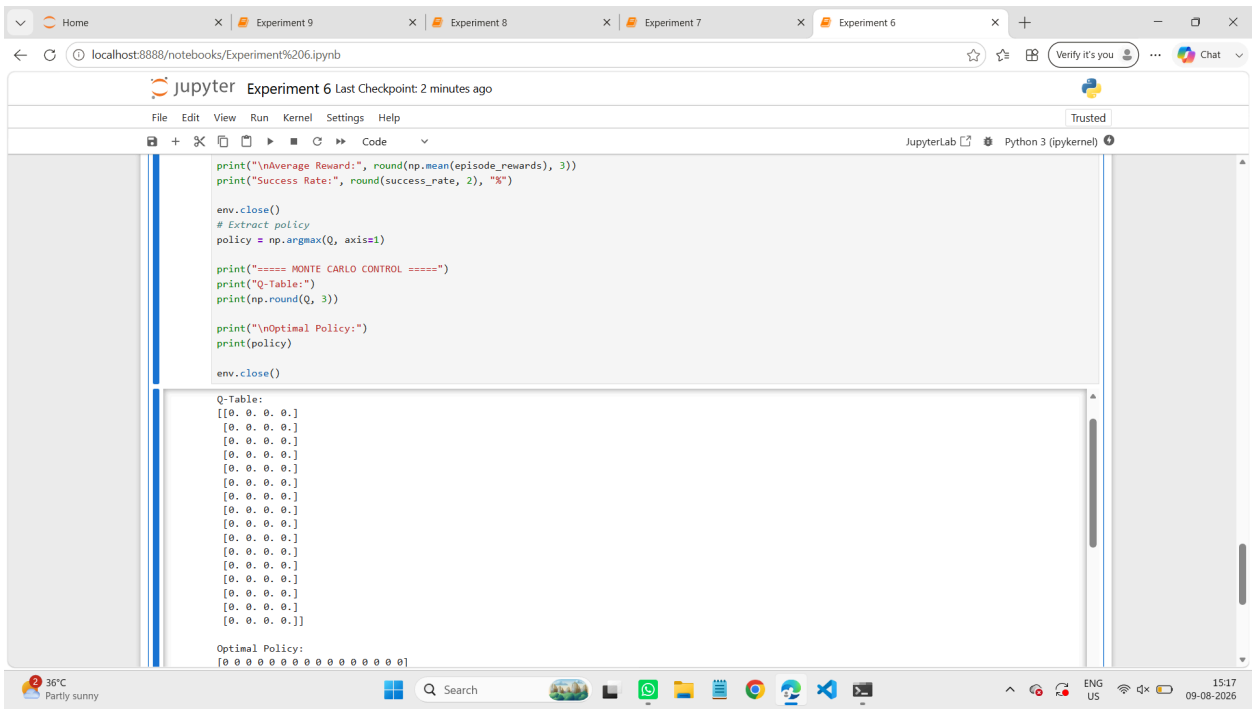
Name: G.Vinay Kumar Reddy **Reg No:**192472093

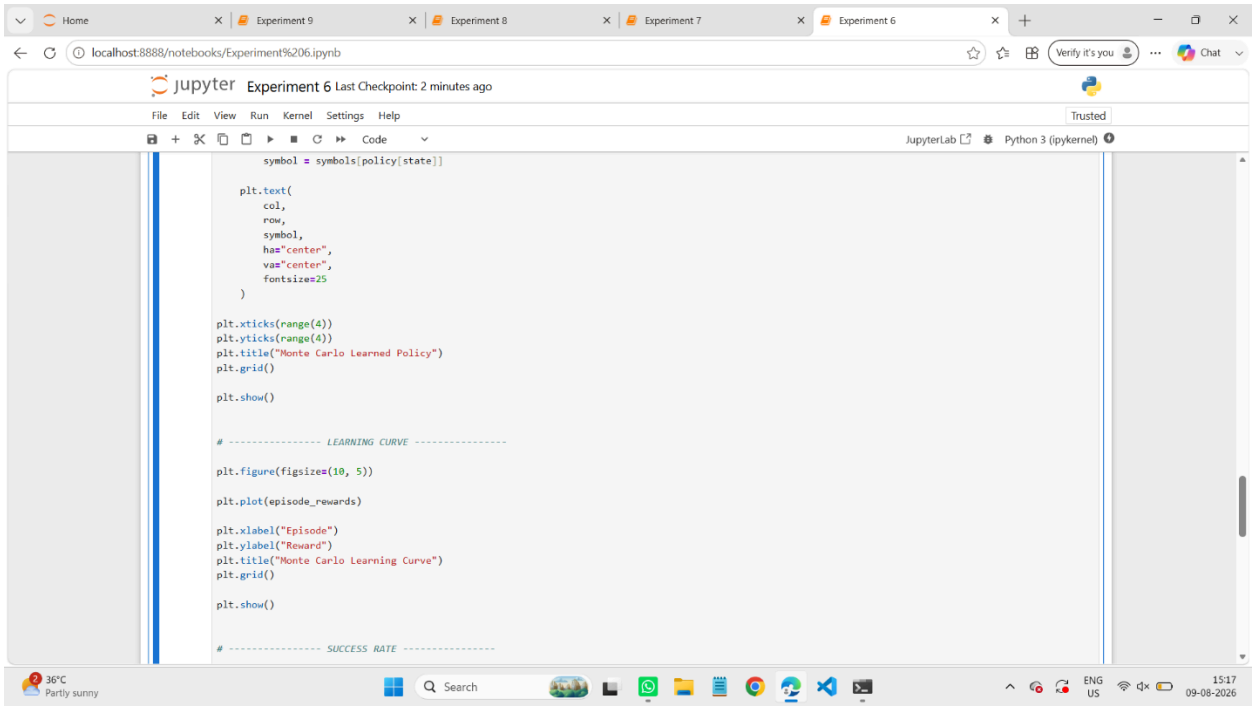
Course Code: MLA0305 **Slot:** B

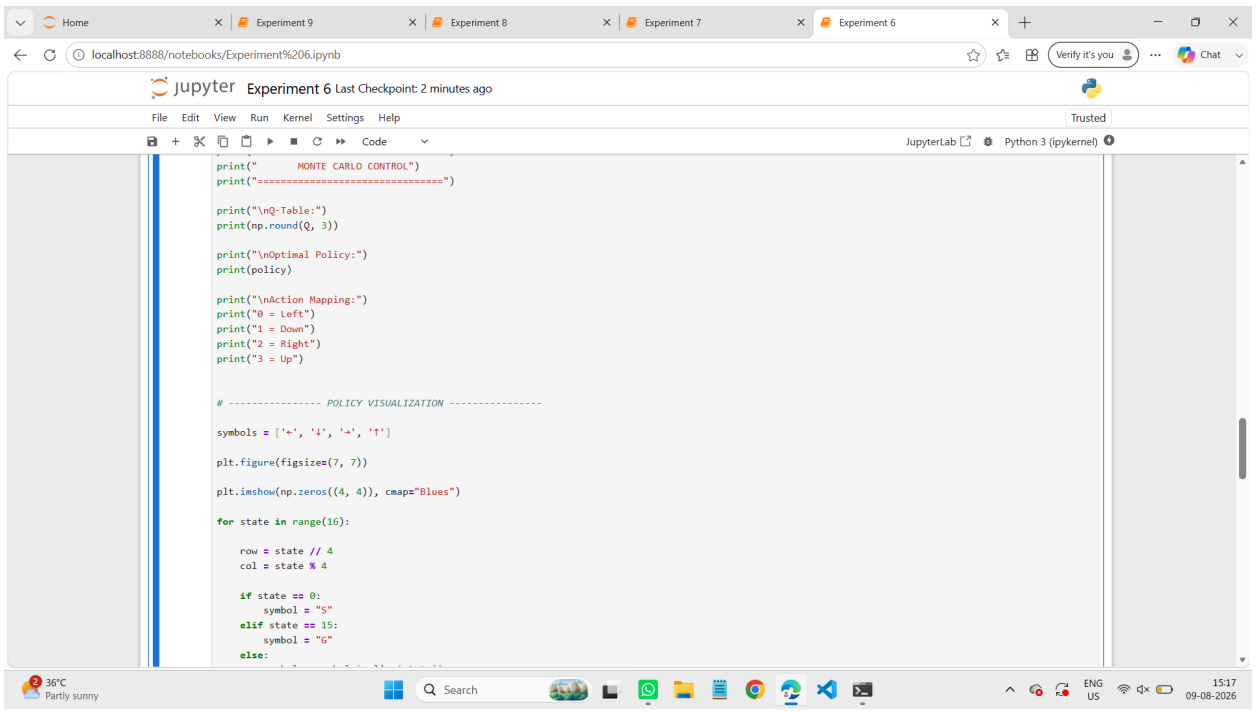


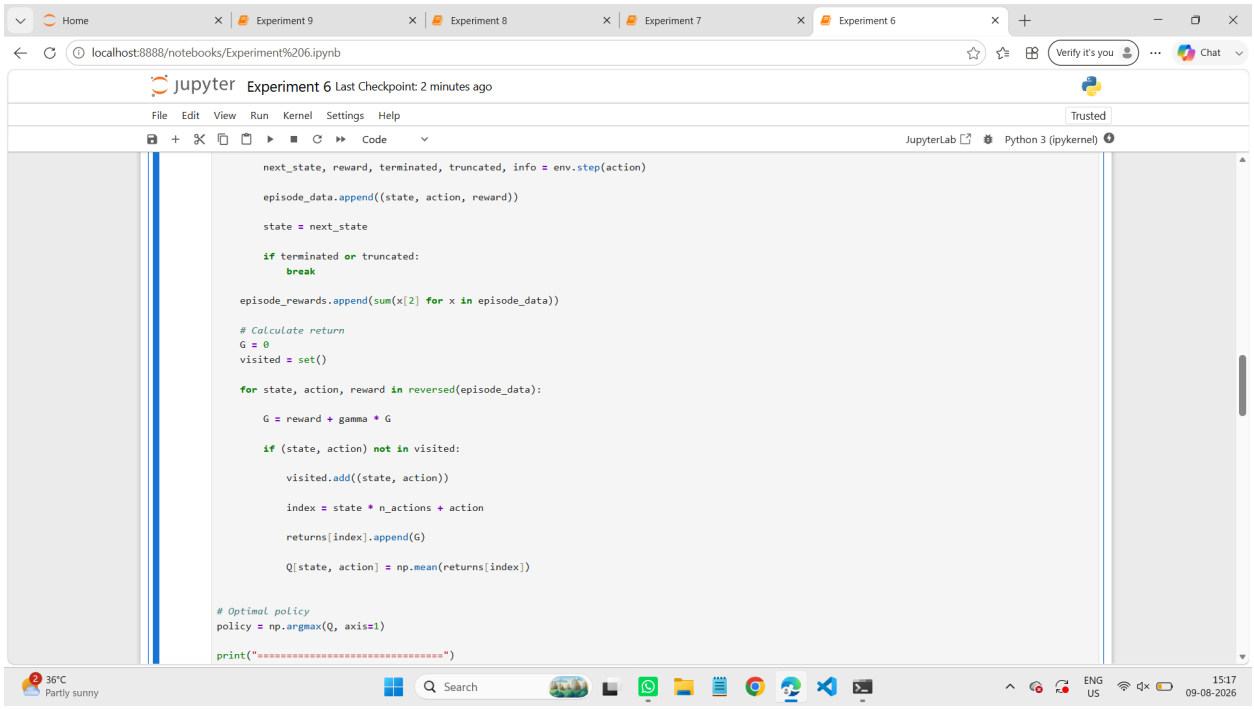



Home
Experiment 9
Experiment 8
Experiment 7
Experiment 6
localhost:8888/notebooks/Experiment%206.ipynb
jupyter Experiment 6 Last Checkpoint: 2 minutes ago
File Edit View Run Kernel Settings Help
Trusted
Code
JupyterLab
Python 3 (ipykernel)
env.close()
Monte Carlo Learned Policy
S
G
0
1
2
3
Search
ENG US
15:18
09-08-2026

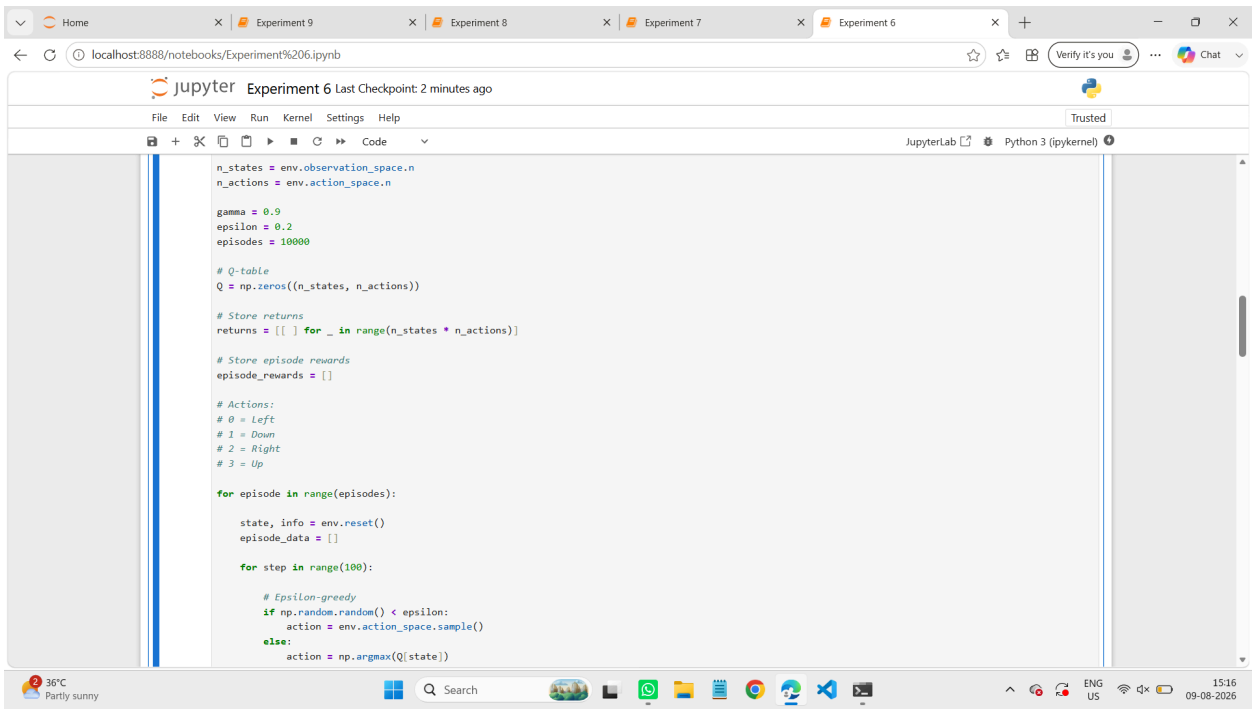

Home
Experiment 9
Experiment 8
Experiment 7
Experiment 6
localhost:8888/notebooks/Experiment%206.ipynb
jupyter Experiment 6 Last Checkpoint: 2 minutes ago
File Edit View Run Kernel Settings Help
Trusted
Code
JupyterLab
Python 3 (ipykernel)
print("\nAverage Reward:", round(np.mean(episode_rewards), 3))
print("Success Rate:", round(success_rate, 2), "%")
env.close()
# Extract policy
policy = np.argmax(Q, axis=1)
print("===== MONTE CARLO CONTROL =====")
print("Q-Table:")
print(np.round(Q, 3))
print("\nOptimal Policy:")
print(policy)
env.close()
Q-Table:
[[0. 0. 0. 0.]
[0. 0. 0. 0.]
[0. 0. 0. 0.]
[0. 0. 0. 0.]
[0. 0. 0. 0.]
[0. 0. 0. 0.]
[0. 0. 0. 0.]
[0. 0. 0. 0.]
[0. 0. 0. 0.]
[0. 0. 0. 0.]
[0. 0. 0. 0.]
[0. 0. 0. 0.]
[0. 0. 0. 0.]
[0. 0. 0. 0.]
[0. 0. 0. 0.]
[0. 0. 0. 0.]]
Optimal Policy:
[0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]
Search
ENG US
15:17
09-08-2026


Home
Experiment 9
Experiment 8
Experiment 7
Experiment 6
localhost:8888/notebooks/Experiment%206.ipynb
jupyter Experiment 6 Last Checkpoint: 2 minutes ago
File Edit View Run Kernel Settings Help
Trusted
Code
JupyterLab
Python 3 (ipykernel)
symbol = symbols[policy[state]]
plt.text(
col,
row,
symbol,
ha="center",
va="center",
fontsize=25
)
plt.xticks(range(4))
plt.yticks(range(4))
plt.title("Monte Carlo Learned Policy")
plt.grid()
plt.show()
# --------------- LEARNING CURVE ---------------
plt.figure(figsize=(10, 5))
plt.plot(episode_rewards)
plt.xlabel("Episode")
plt.ylabel("Reward")
plt.title("Monte Carlo Learning Curve")
plt.grid()
plt.show()
# --------------- SUCCESS RATE ---------------
36°C
Partly sunny
Search
ENG
US
15:17
09-08-2026

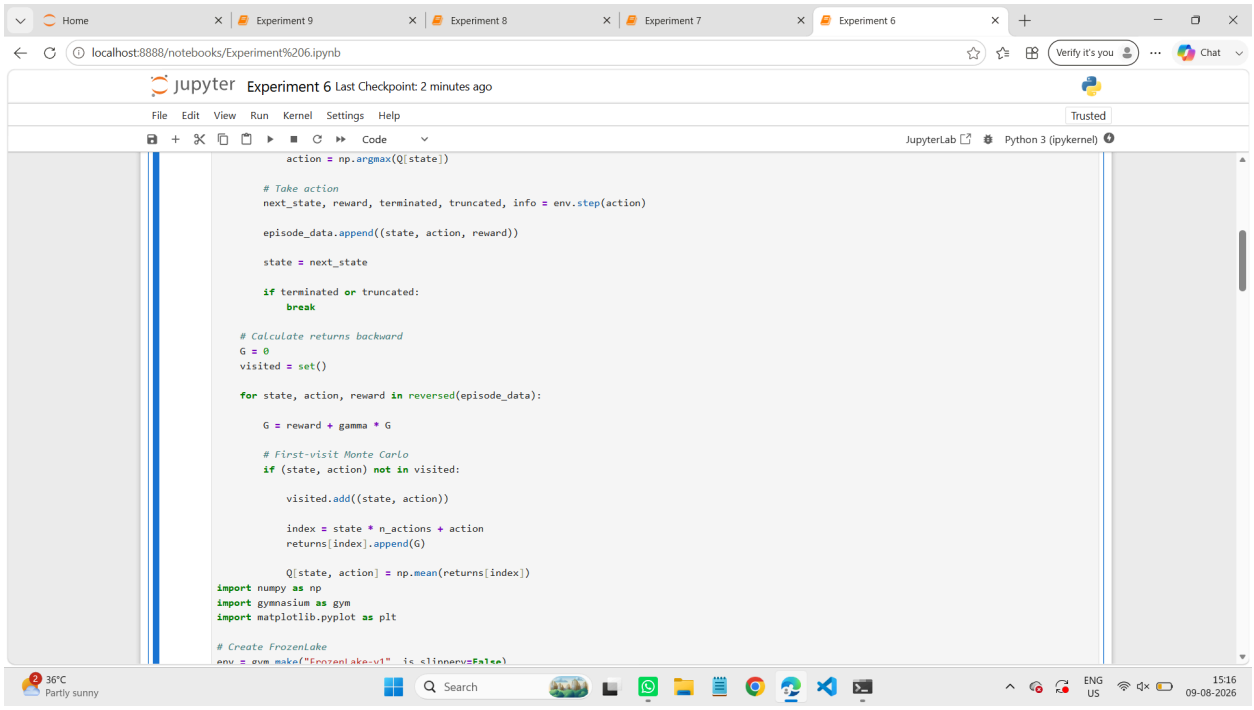

Home
Experiment 9
Experiment 8
Experiment 7
Experiment 6
localhost:8888/notebooks/Experiment%206.ipynb
jupyter Experiment 6 Last Checkpoint: 2 minutes ago
File Edit View Run Kernel Settings Help
Trusted
Code
JupyterLab
Python 3 (ipykernel)
print("   MONTE CARLO CONTROL")
print("==================================")
print("\nQ-Table:")
print(np.round(Q, 3))
print("\nOptimal Policy:")
print(policy)
print("\nAction Mapping:")
print("0 = Left")
print("1 = Down")
print("2 = Right")
print("3 = Up")
# --------------- POLICY VISUALIZATION ---------------
symbols = ['←', '↓', '→', '↑']
plt.figure(figsize=(7, 7))
plt.imshow(np.zeros((4, 4)), cmap="Blues")
for state in range(16):
row = state // 4
col = state % 4
if state == 0:
symbol = "S"
elif state == 15:
symbol = "G"
else:
36°C
Partly sunny
Search
ENG
US
15:17
09-08-2026

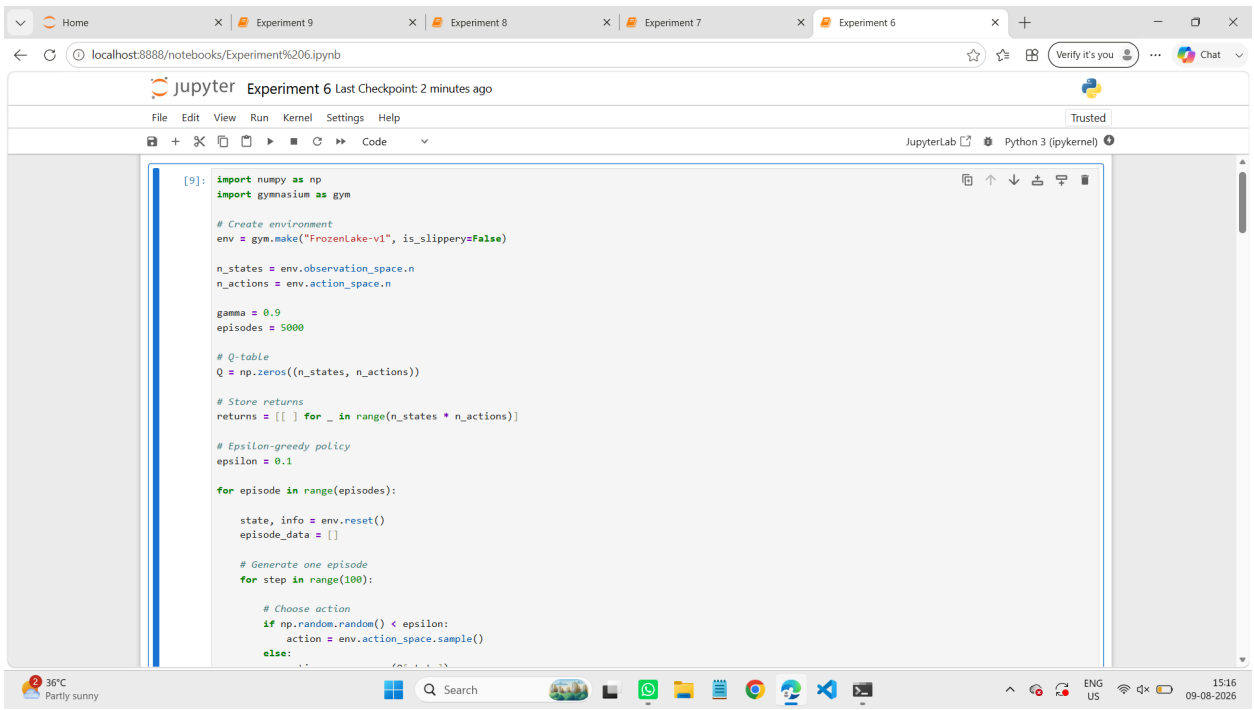


```
        next_state, reward, terminated, truncated, info = env.step(action)

        episode_data.append((state, action, reward))

        state = next_state

        if terminated or truncated:
            break

    episode_rewards.append(sum(x[2] for x in episode_data))

    # Calculate return
    G = 0
    visited = set()

    for state, action, reward in reversed(episode_data):

        G = reward + gamma * G

        if (state, action) not in visited:

            visited.add((state, action))

            index = state * n_actions + action

            returns[index].append(G)

            Q[state, action] = np.mean(returns[index])


# Optimal policy
policy = np.argmax(Q, axis=1)

print("================================")
```



```
n_states = env.observation_space.n
n_actions = env.action_space.n

gamma = 0.9
epsilon = 0.2
episodes = 10000

# Q-table
Q = np.zeros((n_states, n_actions))

# Store returns
returns = [[ ] for _ in range(n_states * n_actions)]

# Store episode rewards
episode_rewards = []

# Actions:
# 0 = Left
# 1 = Down
# 2 = Right
# 3 = Up

for episode in range(episodes):

    state, info = env.reset()
    episode_data = []

    for step in range(100):

        # Epsilon-greedy
        if np.random.random() < epsilon:
            action = env.action_space.sample()
        else:
            action = np.argmax(Q[state])
```


Home
Experiment 9
Experiment 8
Experiment 7
Experiment 6
localhost:8888/notebooks/Experiment%206.ipynb
jupyter Experiment 6 Last Checkpoint: 2 minutes ago
File Edit View Run Kernel Settings Help
Trusted
Code
JupyterLab
Python 3 (ipykernel)
action = np.argmax(Q[state])
# Take action
next_state, reward, terminated, truncated, info = env.step(action)
episode_data.append((state, action, reward))
state = next_state
if terminated or truncated:
break
# Calculate returns backward
G = 0
visited = set()
for state, action, reward in reversed(episode_data):
G = reward + gamma * G
# First-visit Monte Carlo
if (state, action) not in visited:
visited.add((state, action))
index = state * n_actions + action
returns[index].append(G)
Q[state, action] = np.mean(returns[index])
import numpy as np
import gymnasium as gym
import matplotlib.pyplot as plt
# Create FrozenLake
36°C
Partly sunny
Search
ENG
US
15:16
09-08-2026


Home
Experiment 9
Experiment 8
Experiment 7
Experiment 6
localhost:8888/notebooks/Experiment%206.ipynb
jupyter Experiment 6 Last Checkpoint: 2 minutes ago
File Edit View Run Kernel Settings Help
Trusted
Code
JupyterLab
Python 3 (ipykernel)
[9]:
import numpy as np
import gymnasium as gym
# Create environment
env = gym.make("FrozenLake-v1", is_slippery=False)
n_states = env.observation_space.n
n_actions = env.action_space.n
gamma = 0.9
episodes = 5000
# Q-table
Q = np.zeros((n_states, n_actions))
# Store returns
returns = [[ ] for _ in range(n_states * n_actions)]
# Epsilon-greedy policy
epsilon = 0.1
for episode in range(episodes):
state, info = env.reset()
episode_data = []
# Generate one episode
for step in range(100):
# Choose action
if np.random.random() < epsilon:
action = env.action_space.sample()
else:
36°C
Partly sunny
Search
ENG
US
15:16
09-08-2026